Database per Service

own your data; share an API

Watch the 4-min video

In one line: each service owns a private database or schema that only it can read and write; others use its API or events, so schemas can change freely, at the price of no cross-service joins or transactions and some stale copies.

1 One renamed column

Two services share one database. The inventory team renames `qty` to `quantity`, and the order service, which queried that table directly, breaks in production. From then on every schema change needs a meeting. Shared tables are an **untyped, undocumented API** that every team depends on.

2 The rule

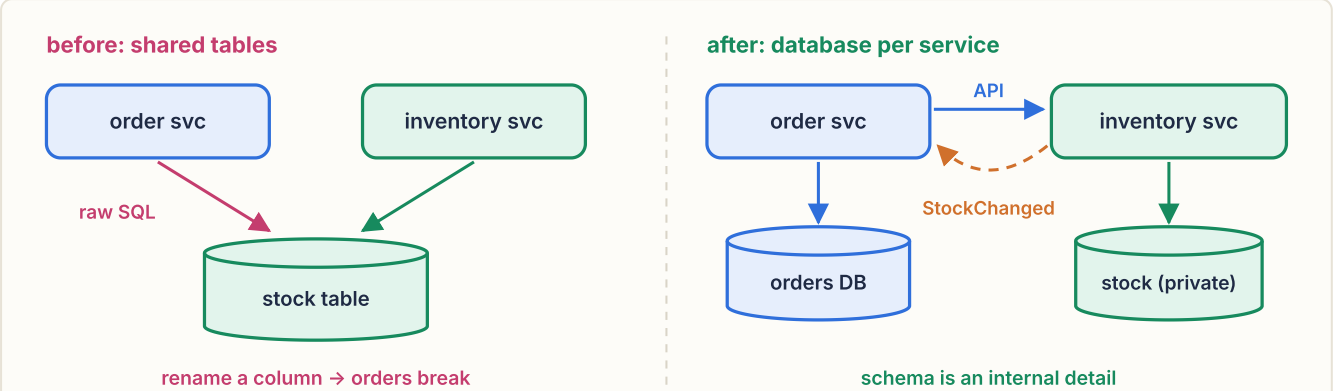


Figure 1. Before: another team's SQL is coupled to your columns. After: only the API and events are the contract.

Each service has a private database or schema. **Nobody else** queries its tables. Other services call its API or consume the events it publishes. The database becomes an implementation detail; the public contract is just the API and the event schemas, which can be versioned.

What you gain: a team can change its schema, indexes, or even its database type (Elasticsearch for search, Postgres for orders, Redis for sessions) without asking anyone. One service's heavy queries no longer slow the others, and each database scales on its own.

3 What it costs

Lost	Replacement
Joins across services	API composition (call both, merge in code) or a local read copy kept fresh by events
Transactions across services	sagas with compensating steps (lesson 34); outbox pattern for reliable events
Always-fresh data everywhere	eventual consistency: copies may be seconds stale
Ad-hoc SQL across all data	a separate analytics platform

To show an order with product names, either call orders and catalog and combine the results, or keep a small `product_name` copy in the order service, updated from `ProductRenamed` events. Read models built this way are the basis of CQRS (lesson 35).

4 Reporting across services



Figure 2. Analytics joins data far from production: services stream changes out, nobody queries their databases.

Don't let reports query every service's database. Each service publishes events, or a **change data capture** tool (such as Debezium, reading the database's write-ahead log) streams its changes into a warehouse or data lake. Analysts join there, and production stays fast.

5 Ownership, not infrastructure

Database per service is about **who may access** the data, not how many servers you run. Several services can share one database server for cost, if each has **its own schema and its own credentials**. Then the rule is enforced: the order service's user simply has no grant on the inventory schema. Without separate credentials it is just a convention, and conventions erode.

WORKED EXAMPLE: INVENTORY EXPOSES STOCK

Before: the order service ran `SELECT qty FROM stock WHERE sku = ?` against inventory's table. After: inventory exposes `GET /stock/{sku}/availability` and `POST /reservations {order_id, sku, qty}`, and publishes `StockChanged` events. At checkout the order service calls the reservation API (with a timeout and an idempotency key on `order_id`). The inventory team then splits `stock` into per-warehouse rows, and no other team notices.

COMMON MISTAKES

- Separate schemas but one shared database user, so anyone can still read anything.
- Reporting jobs reading production databases of every service.
- Chatty API composition on hot paths instead of a local read copy.
- Publishing events outside the database transaction, so they can be lost (use an outbox).

6 Key terms

Database per service: each service's data is private to it

Read copy / replica: local copy of another service's data kept fresh by events

Outbox pattern: write events to a table in the same transaction, then publish

API composition: query several services and join in code

CDC: change data capture: stream DB changes from its log

Integration database: anti-pattern: many apps share one schema

7 Interview questions

Q1. Why shouldn't services share a database? BASIC
Shared tables are a hidden contract: schema changes break other teams, deployments become coupled, and one service's load hurts the others.

Q2. How do other services get data they need? BASIC
Through the owning service's API or by consuming its events, possibly keeping a local read copy.

Q3. How do you report across services? SCENARIO
Stream events or CDC from each service into a warehouse or data lake and join there, never against production databases.

Q4. The order page needs product names from catalog. Options? SCENARIO
API composition at read time, or a local product-name copy in orders updated by catalog events; the copy is faster and survives catalog outages but may be slightly stale.

Q5. Can services share one database server? TRICKY
Yes, for cost, as long as each has its own schema and credentials so only the owner can reach its tables; ownership matters, not hardware.

Q6. How do you update two services' data consistently? TRICKY
Not with a distributed transaction: use a saga with compensations, and the outbox pattern so each step's events are published reliably.

Next: SD-L-006 · Sync vs Async Communication backendroadmap.com